

4장 기계어 : 만들어진 시스템을 분석하면서 어떻게 설계했는지 의도를 파악해라.

*기계어

-기계 수준의 명령어들로 이루어진 저수준 프로그램을 코딩할 수 있게 미리 정의된 규칙이다.

ex: 프로세서의 산술 및 논리연산/ 메모리에서 값 불러오기,적재/ 레지스터간 이동

*기계어는 하드웨어와 소프트웨어가 만나는 접점이자 추상적 기호이다.

명령어는 실리콘 소자 내에서 실행되는 물리적 연산으로 바뀐다.

대부분의 사람들은 기계어로 직접 프로그램을 작성할 일이 없지만, 컴퓨터의 아키텍처를 완전히 이해하려면 저수준 프로그래밍을 공부해야 한다.

*메모리 : 데이터와 메모리 저장하는 하드웨어 장치의 통칭

*프로세서 : CPU

*레지스터 : 메모리 접근 연산은 상대적으로 느리며, 긴 명령어가 필요하다.

주소 하나 표현하는데 32비트가 필요하므로

대부분의 프로세서는 값 하나 저장하는 레지스터 여러 개 두고 있다.

이 레지스터는 프로세서 바로 옆에 위치한다. >> 로컬 고속메모리 역할.

*2진 코드는 암호 같아 보이므로 기계어는 2진 코드와 연상 기호를 같이 사용한다.(ADD)

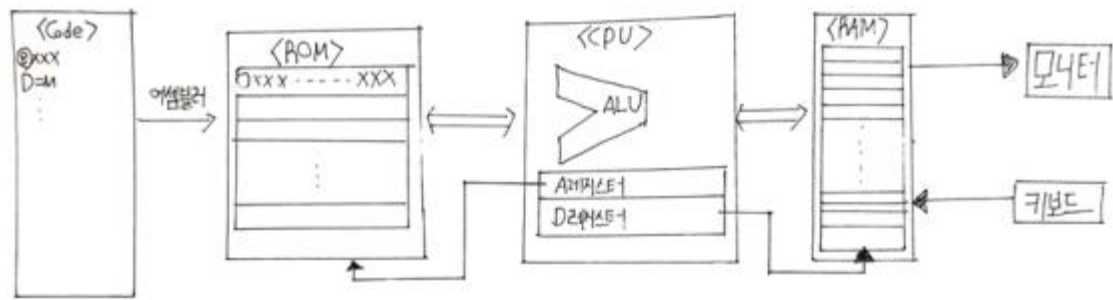
*R1 R3 ADD >> 텍스트 처리프로그램(어셈블러) >> 2진 표현 순으로 진행한다.

이러한 이진 기호 표기법을 어셈블리어

*어셈블리어를 2진 코드로 번역하는 프로그램을 어셈블러 (어셈블러는 기계어의 한 종류)

1.핵 컴퓨터 스펙

- 명령어 메모리 모듈 (16 비트 폭/ 주소비트 15비트) (ROM역할)
(핵의 CPU는 명령어 메모리에 존재하는 프로그램만 수행한다.)
- 데이터 메모리 모듈 (16 비트 폭/ 주소비트 15비트) (RAM역할)
- 스크린/키보드 (16비트 장치)
- CPU
- 폰 노이만 구조



2.레지스터

- 1). D 레지스터 : 데이터 저장하는 역할로 16비트 레지스터 1개이다.
- 2). A 레지스터 : >>16비트 레지스터 1개이다.
>> ROM과 RAM에 해당하는 저장장치와는 다른 것이다.
(레지스터는 메모리 로드 시간이 오래 걸리므로 바로 옆에 둔 저장장치이다.)

3.명령어

- *명령어 1개는 16비트 폭이다.
- *데이터 또한 16비트 폭이므로 주소와 데이터를 한 번에 명령어에 넣을 수 없다.

[M 명령어]

이를 해결하기 위해 M이라는 이름의 주소로만 메모리 접근이 가능하도록 강제한다.

M=Memory[A] 라고 보면 된다.

(M=RAM[A], ROM[A] 둘 다 가능하다. ROM[A]는 A번째 ROM레지스터에 있는 명령을 의미)

- *명령어는 2가지로 나눌 수 있다. A명령어/C명령어

1). A-명령어

A 레지스터에 15 비트값을 설정하는 데 쓰인다. (레지스터에 값 저장용)

* @value 명령어

-@value라는 명령어는 A 레지스터에 0xxx_xxxx_xxxx_xxxx 저장한다.

ex)@5는 0000_0000_0000_0101을 A 레지스터에 저장한다.

-이 명령어는 1)@constant 2)@symbol 2가지 용도로 쓸 수 있다.

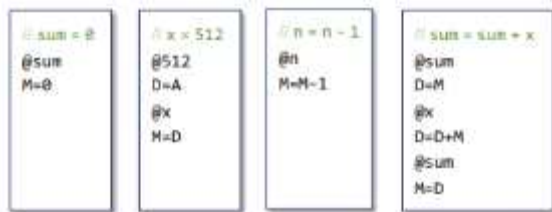
(1)@constant

value라는 값을 A 레지스터에 저장하라는 것

ex) @100 // A 레지스터에 100 넣는다.

(2)@symbol

RAM의 레지스터 중 하나를 임의로 선정하여 그 주소를 A레지스터에 담는다.

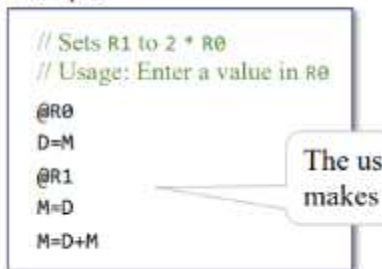


*미리 정의된 symbol :R0 ~ R15

symbol	value	RAM
R0	0	0 R0
R1	1	1 R1
R2	2	2 R2
...	...	...
R15	15	15 R15
		16
		17
		...
		32767

R0 주소 0부터 R15 주소 15까지 나타낸다.

Example:



@R0 하면 A에 0 넣는 것과 같다.

2). C-명령어

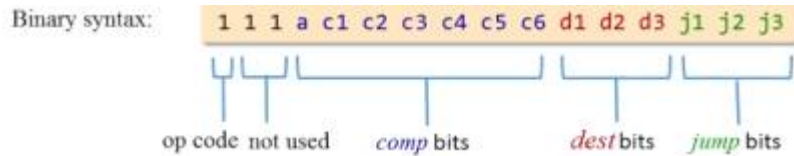
연산을 수행하는 명령어

C-명령어는 dest=comp;jmp 구조이다.(저장장소/연산식/점프식)

C-명령어의 이진코드 = 111a cccc_ccdd_djjj 형태이다.

(?? C-명령어의 이진코드는 어디에 저장하는가?)

>> 명령어 메모리에 저장한다. 텍스트 파일에 있는 기계어 프로그램을 명령어 메모리로 로드하는 기능 제공.



(when a=0) <i>comp mnemonic</i>	c1	c2	c3	c4	c5	c6	(when a=1) <i>comp mnemonic</i>
0	1	0	1	0	1	0	
1	1	1	1	1	1	1	
-1	1	1	1	0	1	0	
D	0	0	1	1	0	0	
A	1	1	0	0	0	0	M
!D	0	0	1	1	0	1	
!A	1	1	0	0	0	1	!M
-D	0	0	1	1	1	1	
-A	1	1	0	0	1	1	-M
D+1	0	1	1	1	1	1	
A+1	1	1	0	1	1	1	M+1
D-1	0	0	1	1	1	0	
A-1	1	1	0	0	1	0	M-1
D+A	0	0	0	0	1	0	D+M
D-A	0	1	0	0	1	1	D-M
A-D	0	0	0	1	1	1	M-D
D&A	0	0	0	0	0	0	D&M
D A	0	1	0	1	0	1	D M

d1	d2	d3	Mnemonic	Destination (where to store the computed value)
0	0	0	null	The value is not stored anywhere
0	0	1	M	Memory[A] (memory register addressed by A)
0	1	0	D	D register
0	1	1	MD	Memory[A] and D register
1	0	0	A	A register
1	0	1	AM	A register and Memory[A]
1	1	0	AD	A register and D register
1	1	1	AMD	A register, Memory[A], and D register

j1 (out < 0)	j2 (out = 0)	j3 (out > 0)	Mnemonic	Effect
0	0	0	null	No jump
0	0	1	JGT	If out > 0 jump
0	1	0	JEQ	If out = 0 jump
0	1	1	JGE	If out ≥ 0 jump
1	0	0	JLT	If out < 0 jump
1	0	1	JNE	If out ≠ 0 jump
1	1	0	JLE	If out ≤ 0 jump
1	1	1	JMP	Jump

-C명령어 예제

<pre>// D ← 2 D=1 D=D+1</pre>	<pre>// D ← 1954 @1954 D=A</pre>	<pre>// D ← D + 23 @23 D=D+A</pre>
<pre>// RAM[100] ← 0 @100 M=0</pre>	<pre>// RAM[100] ← 17 @17 D=A @100 M=D</pre>	<pre>// RAM[100] ← RAM[200] @200 D=M @100 M=D</pre>
<pre>// RAM[3] ← RAM[3] - 15 @15 D=A @3 M=M-D</pre>	Use only the instructions shown above	<pre>// RAM[3] ← RAM[4] + 1 @4 D=M+1 @3 M=D</pre>

17같은 아무 상수를 더하는 것은 안된다. 오직 1하고 -1만 더할 수 있다.

```
// Computes: RAM[2] = RAM[0] + RAM[1] + 17
// D ← RAM[0]
@0
D=M
// D ← D + RAM[1]
@1
D=D+M
// D ← D + 17
@17
D=D+A
// RAM[2] ← D
@2
M=D
```

```
// if (RAM[3] < 100) goto 12
// D ← RAM[3] - 100
@3
D=M
@100
D=D-A
// if (D < 0) goto 12
@12
D;JLT
```

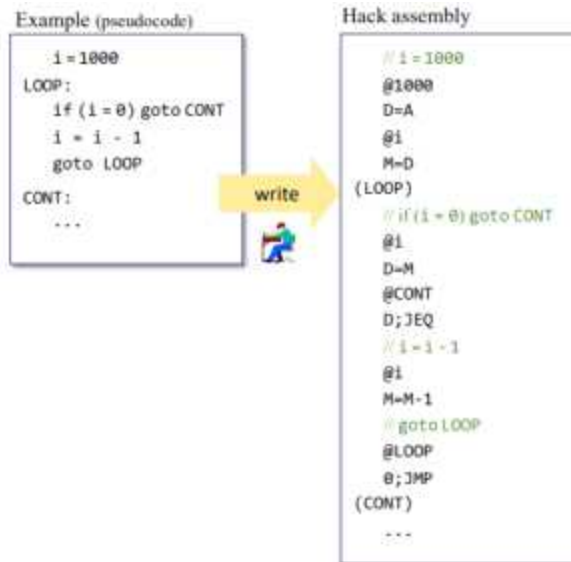
여기서 goto12라는 것은 ROM의 12번째 줄에 있는 명령 수행하라는 것이다.

@12부분을 보면 A에 12넣은 것이고 점프 명령어는 A어드레스를 이용하여 ROM에 접근한다.

(점프 명령어는 RAM에 접근하는 것이 아니다.)

4. 라벨

(LOOP)같은 것이라고 보면 된다. 코드를 보면서 이해하라.



@LOOP는 A 어드레스에 (LOOP)가 위치한 줄 번호를 넣는다.

즉, 그 다음에 오는 점프 함수를 통해 다시 (LOOP)위치로 돌아가라는 것.

@CONT도 같은 논리이다.

```
// Sets R2 to R0 + R1 + 17
// D = R0
@R0
D=M
// D = D + R1
@R1
D=D+M
// D = D + 17
@17
D=D+A
// R2 = D
@R2
M=D
```

Pseudocode

```
// if R0 >= 0 then R1 = 1
// else R1 = -1
if (R0 >= 0) goto POS
R1 = -1
goto END
POS:
R1 = 1
END:
```

Signum.asm

```
// if R0 >= 0 then R1 = 1
// else R1 = -1
// if R0 >= 0 goto POS
@R0
D=M
@POS
D;JGE
// R1 = -1
@R1
M=-1
// goto END
@END
0;JMP
(POS)
// R1 = 1
@R1
M=1
(END)
```

The assembler replaces all the symbols with physical addresses

Assembler / loader

(the assembler generates binary instructions; Here we show their symbolic versions, for readability)

Memory

0	@0
1	D=M
2	@8
3	D;JGE
4	@1
5	M=-1
6	@10
7	0;JMP
8	@1
9	M=1
10	
11	
12	
13	
14	
...	

위 코드를 구동한다고 했을 때, ROM에서 0번부터 9번 레지스터만 읽는 것이 아니다. ROM에서는 0번부터 끝까지 전부 읽어버리므로 끝나려면 오래 걸린다. 또한, 쓰레기 값이 있는 레지스터도 읽으므로 문제가 된다. 따라서 구동하고자 하는 부분만 읽을 수 있도록 해주어야 한다. 이를 위해 Infinite Loop를 이용한다.

Pseudocode

```
// if R0 >= 0 then R1 = 1
// else R1 = -1
if (R0 >= 0) goto POS
R1 = -1
goto END
POS:
R1 = 1
END:
```

Signum.asm

```
// if R0 >= 0 then R1 = 1
// else R1 = -1
// if R0 >= 0 goto POS
@R0
D=M
@POS
D;JGE
// R1 = -1
@R1
M=-1
// goto END
@END
0;JMP
(POS)
// R1 = 1
@R1
M=1
(END)
@END
0;JMP
```

Infinite loop

Example: Compute $1 + 2 + 3 + \dots + N$

Pseudocode

```
// Program: Sum1ToN (R0 represents N)
// Computes R1 = 1 + 2 + 3 + ... + R0
// Usage: put a value >= 1 in R0
i = 1
sum = 0
LOOP:
  if (i > R0) goto STOP
  sum = sum + i
  i = i + 1
  goto LOOP
STOP:
  R1 = sum
```

Hack assembly

```
// Program: Sum1ToN (R0 represents N)
// Computes R1 = 1 + 2 + 3 + ... + R0
// Usage: put a value >= 1 in R0
// i = 1
@i
M=1
// sum = 0
@sum
M=0
(LLOOP)
  // if (i > R0) goto STOP
  @i
  D=M
  @R0
  D=D-M
  @STOP
  D;JGT
  // sum = sum + i
  @sum
  D=M
  @i
  D=D+M
  @sum
  M=D
  // i = i + 1
  @i
  M=M+1
  // goto LOOP
@LLOOP
0;JMP
```

(code continues here)

```
(STOP)
  // R1 = sum
  @sum
  D=M
  @R1
  M=D
  // infinite loop
(END)
  @END
  0;JMP
```


5. 포인터 및 배열 프로그래밍

1)포인터

Example 2: Get the value of the register at address *addr*

Input: R0 holds *addr*

```
// Gets R1 = RAM[R0]
// Usage: Put some non-negative value in R0
@R0
A=M
D=M
@R1
M=D
```

RAM		
0	1013	R0
1	75	R1
2		R2
...		...
15		R15
16		
17		
...		
255		
256		
...		
1012	512	
1013	75	
1014	19	
1015	-17	
1016	256	
...		

example:
addr = 1013

2)배열

High-level code (Java example)

```
...
// Variable declarations
int[] arr = new int[5];
int sum = 0;
...
// Enters some values into the array
// (code omitted)
...
// Sums up the array elements
for (int j=0; j<5; j++) {
    sum = sum + arr[j];
}
...
// Increments each array element
for (int j=0; j<5; j++) {
    arr[j] = arr[j]+1
}
...
```

Compiler

Hack assembly

```
...
// sum = sum + arr[j]
@arr
D=M
@j
A=D+M
D=M
@sum
M=M+D
...
// arr[j] = arr[j] + 1
@arr
D=M
@j
A=D+M
M=M+1
...
```

6. 스크린

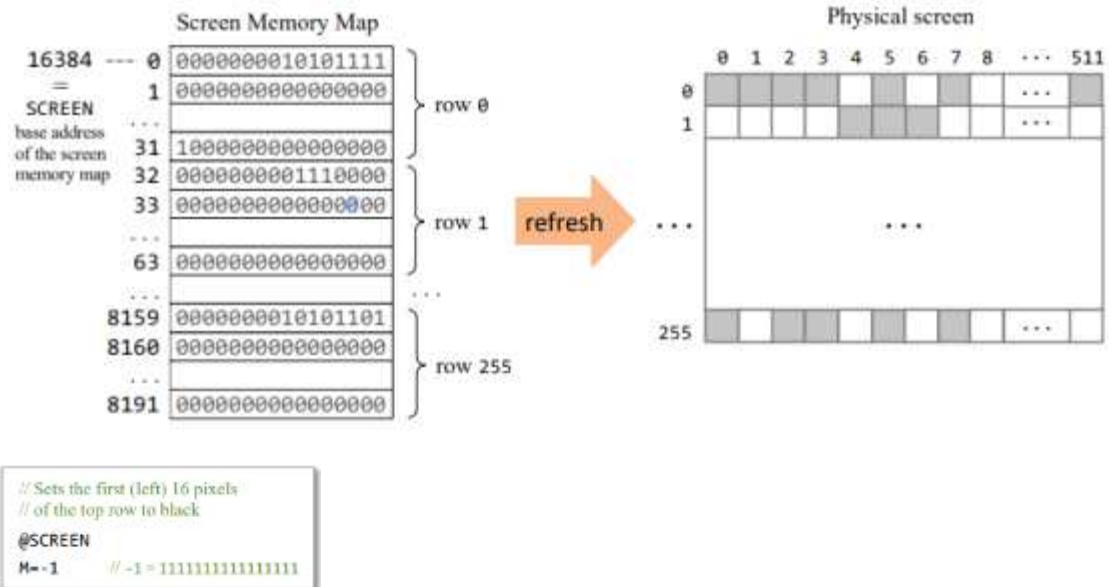
- 스크린은 행 256 / 열 512개의 픽셀로 구성된다.
- 스크린의 픽셀값을 저장하는 위치는 RAM의 16384번 레지스터부터 시작이다.
(RAM의 레지스터는 16비트인 워드 1개이다.)
- r 번째 행의 c 열의 픽셀은 $\text{RAM}[16384+r*32+c/16]$ 에 위치한 레지스터의 $c\%16$ 번째 비트와 대응한다.
- 16384번 레지스터에서 시작하여 32개의 레지스터가 모니터의 1 횡씩 가져간다.

pf) r 행의 c 열에 위치한 픽셀은 $r*512+c$ 번째 픽셀이다.

$\text{RAM}[\lfloor (16384+(r*512+c))/16 \rfloor]$ 에 위치한 레지스터에 대응하는 비트가 있을 것이다.

열은 $(r*512+c)\%16$ 이다. $\{(r*512+c)\%16\}=c\%16$ 이다.

그래서 위의 식이 나오는 것이다.



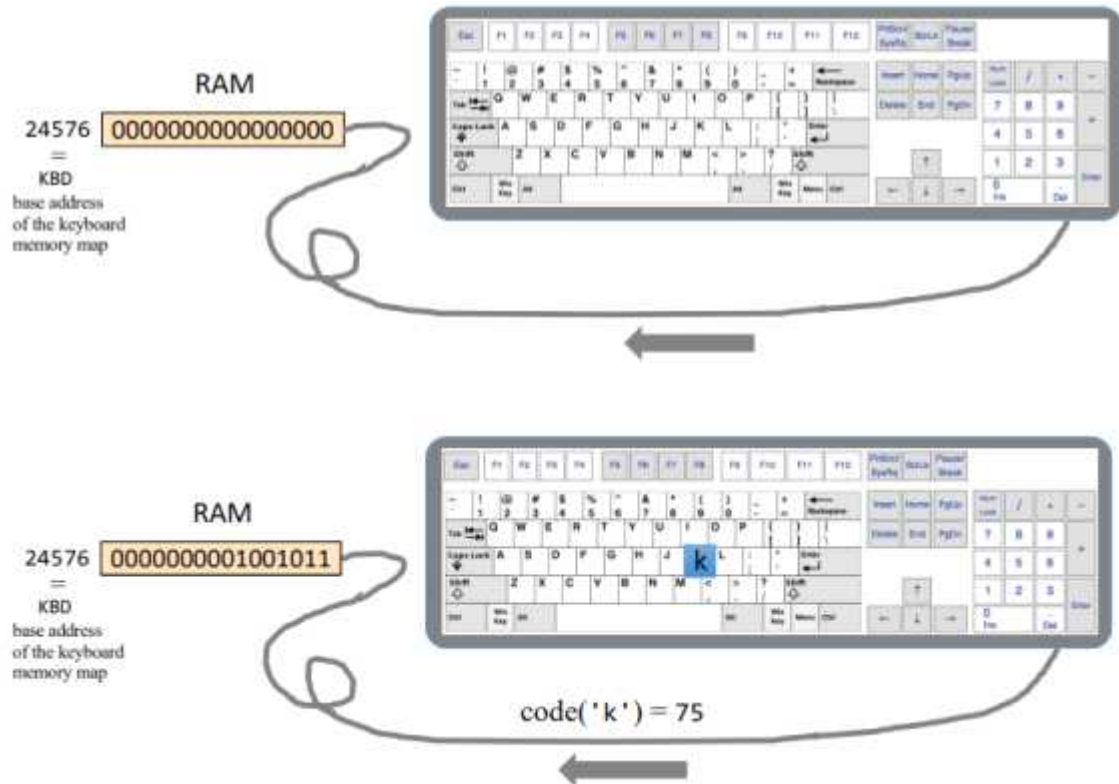
그냥 레지스터의 비트를 설정하기만 하면 된다.

7.키보드

key	code	key	code	key	code	key	code	key	code
(space)	32	0	48	A	65	a	97	newline	128
!	33	1	49	B	66	b	98	backspace	129
"	34	...	...	C	...	c	99	left arrow	130
#	35	9	57	...	...	...	...	up arrow	131
\$	36	:	58	Z	90	z	122	right arrow	132
%	37	;	59	[	91	{	123	down arrow	133
&	38	<	60	/	92		124	home	134
'	39	=	61	]	93	}	125	end	135
(	40	>	62	^	94	~	126	Page up	136
)	41	?	63	_	95			Page down	137
*	42	@	64	`	96			insert	138
+	43							delete	139
,	44							esc	140
-	45							f1	141
.	46							...	...
/	47							f12	152

(Subset of Unicode)

키보드의 Base 주소는 24576



Examples:

```
// Set D to the character code of
// the currently pressed key
@KBD
D=M
```

```
// If the currently pressed key is 'q', goto END
@KBD
D=M
@113 // 'q'
D=D-A
@END
D;JEQ
```

8.built in symbol

symbol	value	symbol	value
R0	0	SP	0
R1	1	LCL	1
R2	2	ARG	2
...	...	THIS	3
R15	15	THAT	4
SCREEN	16384		
KBD	24576		

9.과제

이진수 사칙연산을 연습할 필요가 있다

<Mult>

//곱셈을 구현하는데 시프트 연산도 쓰고 다른 방법도 있겠지만

//a*b는 결국 a를 b번 더하는 것이다.

//더하기만을 이용해서 구현한다.

//R2 초기값은 0이다

@R2

M=0

//R0는 0이상이다.

@R0

D=M

@END

D;JEQ

//R1는 0이상이다.

@R1

D=M

@END

D;JEQ

//R3는 변수로 사용한다. R0의 값을 담는다.

@R0

D=M

@R3

M=D

//곱셈한다.

(LOOP)

@R1

D=M

@R2

M=M+D

@R3

M=M-1

D=M

@LOOP

D;JGT

(END)

@END

0;JMP

<Fill>

//Preset: R0스크린 시작값, R0는 뺄두고 R1에서 쓸거다.

@SCREEN

D=A

@0

M=D

(Reset)

//R1값 세팅

@0

D=M

@1

M=D

//키보드값을 받는다. 시작부

(GetVal)

@KBD

D=M

@Black

D;JGT

@White

D;JEQ

@GetVal

0;JMP

//키보드값 받는다 종점부

(White)

//0. 화면 전부 하얗게 한다.

//R1에 있는 값을 주소로하는 레지스터 0할당

@1

A=M

M=0

//R1 값 1증가

@1

M=M+1

//R1에 있는 값이 24575보다 크다면 Reset으로가

@1

D=M

@24575

D=D-A

@Reset

D;JGT

//R1에 있는 값이 24575보다 작거나 같다면 White으로가

@White

D;JLE

(Black)

//0. 화면 전부 검게 한다.

//R1에 있는 값을 주소로하는 레지스터 -1할당

@1

A=M

M=-1

//R1 값 1증가

@1

M=M+1

//R1에 있는 값이 24575보다 크다면 Reset으로가

@1

D=M

@24575

D=D-A

@Reset

D;JGT

//R1에 있는 값이 24575보다 작거나 같다면 Black으로가

@Black

D;JLE